

DP Computer Science: B 4.1.6 Core Principles of ADTs (Python) (HL Only)

1. Core Concepts & Learning Objectives

Overview

These lessons focus on Abstract Data Types (ADTs) and Hash Tables/Maps. They cover core concepts like **key-value pairs**, **hashing functions**, **collision resolution** (linear probing and chaining), **load factor**, and practical applications like **word frequency counting**.

Key Vocabulary

Term	Definition
ADT (Abstract Data Type)	A high-level description of a data structure, specifying operations without implementation details
Hash Table	A data structure that stores key-value pairs and provides O(1) average-time lookups
Key	The unique identifier used to store and retrieve a value
Value	The data associated with a key
Hashing Function	A function that converts a key into an array index
Hash Code	The integer produced by a hashing function
Collision	When two different keys hash to the same array index
Linear Probing	A collision resolution strategy that finds the next available slot

Term	Definition
Chaining	A collision resolution strategy that stores multiple items in a list at each slot
Load Factor	The ratio of items to slots ($\alpha = \text{items} / \text{table size}$)
Rehashing	Creating a larger table and reinserting all items when the load factor is too high
Dictionary	Python's implementation of a hash table

ATL Skills Focus

Skill	Description
Thinking (Abstraction)	Grasping the concept of an ADT, separating "what" (operations) from "how" (implementation)
Thinking (Algorithmic Thinking)	Following the logical steps of hashing and collision resolution
Thinking (Application & Synthesis)	Synthesising knowledge to solve practical problems
Communication	Using precise terminology like ADT, key, value, hash table
Self-Management	Reflecting on performance implications; structuring programs

2. Lesson 1: Introduction to ADTs and Hash Tables

Learning Intentions

By the end of this lesson, students will be able to:

- Explain the concept of an **Abstract Data Type (ADT)** as a high-level description of a data structure
- Describe the purpose of a **hash table** (dictionary/map) for storing and retrieving **key-value pairs**
- Explain why hash tables provide **fast lookups, insertions, and deletions** compared to lists
- Use a Python **dictionary** (`dict`) to store and access key-value data

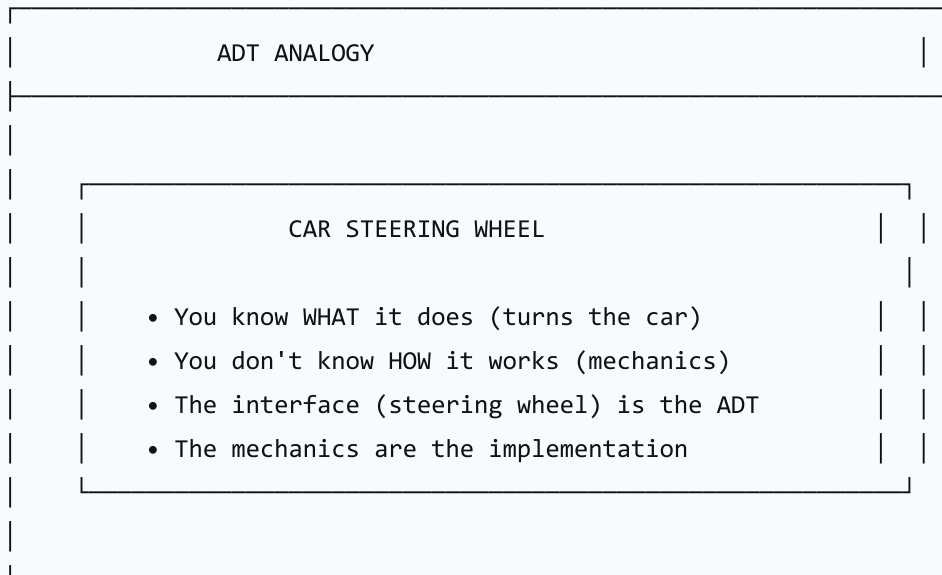
The Problem with Lists

"If you had a list of 1 million student records and needed to find one student by their unique ID, you would need to search through the list until you find it. This is $O(n)$ — too slow for frequent lookups."

What is an ADT?

An Abstract Data Type (ADT) is a logical description or blueprint for a data structure. It specifies *what* operations can be performed (add, remove, find) but not *how* they are implemented.

text



Key-Value Pairs and Hash Tables

text

KEY-VALUE PAIRS	
KEY	VALUE
"apple"	"A red fruit"
"dog"	"A furry animal"
"student_123"	"Alice Smith"
"max_score"	100

A Hash Table stores these key-value pairs, providing fast lookups by key.

Python Dictionaries

python

Creating a dictionary

```
student = {
    "name": "Alice Smith",
    "age": 17,
    "grade": 12,
    "subjects": ["Math", "Science", "English"]
}
```

Accessing values by key

```
print(student["name"])    # Output: Alice Smith
print(student["age"])     # Output: 17
```

Adding a new key-value pair

```
student["email"] = "alice@school.com"
```

Modifying an existing value

```
student["grade"] = 11
```

Checking if a key exists

```

if "email" in student:
    print("Email exists!")

# Getting a value with a default (if key doesn't exist)
phone = student.get("phone", "No phone number")

# Iterating over keys and values
for key, value in student.items():
    print(f"{key}: {value}")

```

List vs. Dictionary Performance

Operation	List	Dictionary (Hash Table)
Lookup by key/index	O(n) (search)	O(1) average
Insertion	O(1) (at end)	O(1) average
Deletion	O(n)	O(1) average
Ordered	Yes	No (Python 3.7+ preserves insertion order but not sorted)

3. Lesson 2: Hashing Functions

Learning Intentions

By the end of this lesson, students will be able to:

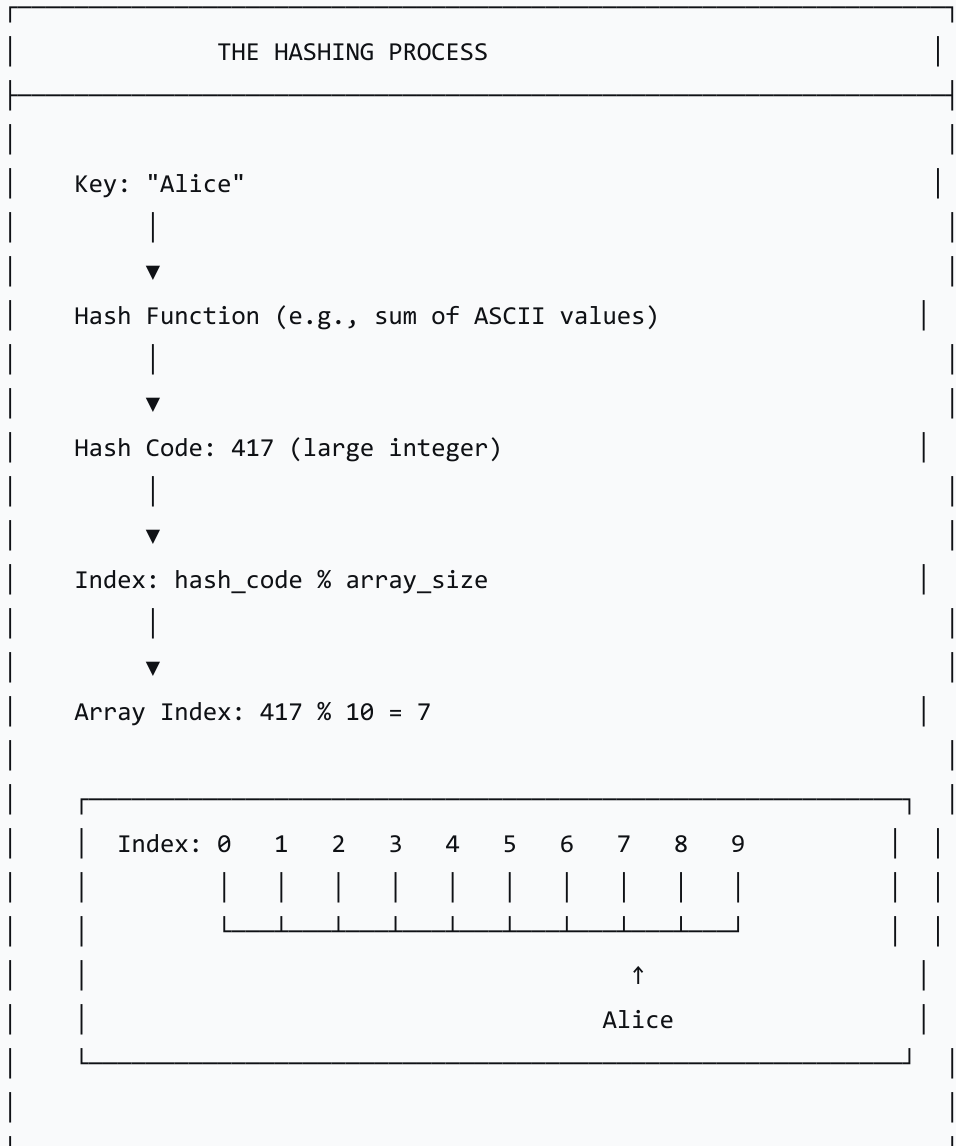
- Explain that a hash table is internally just an **array**
- Define a **hashing function** and its purpose: to convert a key into an array index
- Describe the properties of a **"good" hashing function** (deterministic, uniform distribution)
- Trace a simple hashing function to find the index for a given key

How Does a Hash Table Work?

"A hash table is just an array! The 'magic' is converting a key (like 'Alice') into an array index (like 3). This is the job of the hashing function."

The Hashing Process

text



Properties of a Good Hashing Function

Property	Description
Deterministic	The same key must always produce the same hash code
Uniform Distribution	The function should spread keys evenly across array slots to avoid clustering
Fast to Compute	Should be quick to calculate
Minimises Collisions	Different keys should ideally produce different hash codes

Simple Hashing Example

python

```
# Simple hash function: sum of ASCII values of characters
def simple_hash(key, table_size):
    total = 0
    for char in key:
        total += ord(char) # ord() returns the ASCII value
    return total % table_size

# Examples
print(simple_hash("cat", 10)) # ord('c')=99 + ord('a')=97 + ord('t')=116 = 312 % 10 = 2
print(simple_hash("dog", 10)) # 100 + 111 + 103 = 314 % 10 = 4
print(simple_hash("car", 10)) # 99 + 97 + 114 = 310 % 10 = 0
```

Poor Hashing Function Example

python

```
# Bad hash function: uses string length
def bad_hash(key, table_size):
    return len(key) % table_size

# Problem: "cat" and "dog" both have length 3, so they hash to the same index!
# This causes many collisions.
```

4. Lesson 3: Collision Resolution

Learning Intentions

By the end of this lesson, students will be able to:

- Define a **hash collision**
- Explain why hash collisions are **practically unavoidable**
- Describe and trace **linear probing** as a collision resolution strategy
- Describe and trace **chaining** as a collision resolution strategy
- Sketch the state of a hash table after inserting items that cause collisions

Hash Collisions

A hash collision occurs when two different keys hash to the same array index.

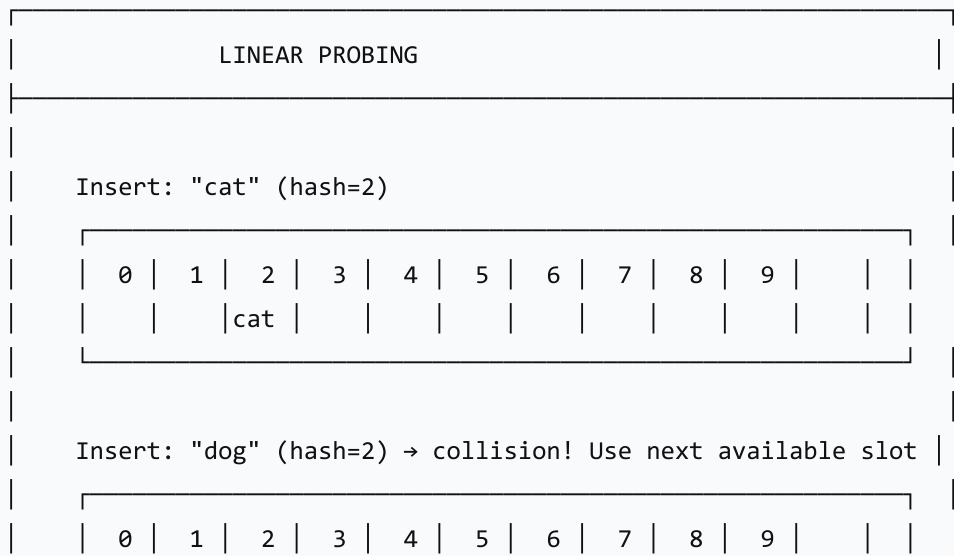
Why Collisions Are Unavoidable

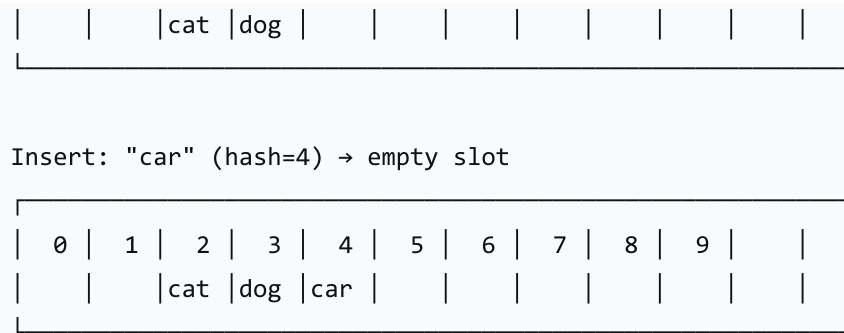
"The pigeonhole principle: if you have more keys than array slots, a collision must happen."

Collision Resolution Strategy 1: Linear Probing

"If my slot is taken, just try the next one. If that's taken, try the next one, and so on, wrapping around if necessary."

text





Linear Probing Code

python

```
class LinearProbingHashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * size

    def hash_function(self, key):
        total = sum(ord(c) for c in key)
        return total % self.size

    def insert(self, key, value):
        index = self.hash_function(key)

        # Linear probing
        while self.table[index] is not None and self.table[index][0] != key:
            index = (index + 1) % self.size
            if index == self.hash_function(key): # Table is full
                raise Exception("Hash table is full")

        self.table[index] = (key, value)

    def search(self, key):
        index = self.hash_function(key)

        while self.table[index] is not None:
            if self.table[index][0] == key:
                return self.table[index][1]
            index = (index + 1) % self.size
```

```

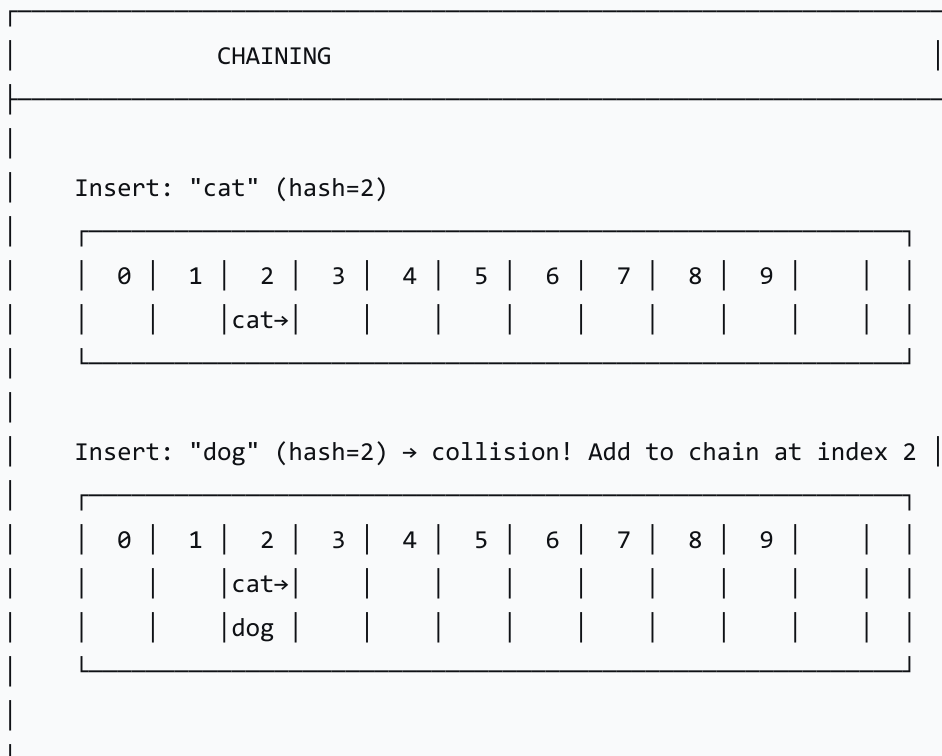
        if index == self.hash_function(key):
            break
    return None

```

Collision Resolution Strategy 2: Chaining

"Each slot in our array isn't a single item, it's a list (like a chain) of items. If a collision occurs, just append the new item to the list at that index."

text



Chaining Code

python

```

class ChainingHashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)] # Each slot is an empty list

    def hash_function(self, key):

```

```
total = sum(ord(c) for c in key)
return total % self.size

def insert(self, key, value):
    index = self.hash_function(key)

    # Check if key already exists (update)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            self.table[index][i] = (key, value)
            return

    # Key not found, add new entry
    self.table[index].append((key, value))

def search(self, key):
    index = self.hash_function(key)

    for k, v in self.table[index]:
        if k == key:
            return v
    return None
```

5. Lesson 4: Load Factor, Rehashing, and Applications

Learning Intentions

By the end of this lesson, students will be able to:

- Define **load factor** and calculate it for a given hash table
 - Explain how a high load factor impacts performance and leads to **rehashing**
 - Explain that Python's `set` is also implemented using a hash table
 - Write a Python program that uses a `dict` to solve a practical problem
-

Load Factor

text

LOAD FACTOR
Load Factor (α) = Number of Items / Number of Slots
Example:
• 5 items in a table of size 10 $\rightarrow \alpha = 0.5$ • 8 items in a table of size 10 $\rightarrow \alpha = 0.8$
Low load factor (< 0.5): Few collisions, fast lookups
High load factor (> 0.75): Many collisions, slower lookups

Rehashing

"When the load factor gets too high (e.g., > 0.75), the hash table is rehashed: a new, bigger array is created, and all existing items are inserted again using the new size."

python

```
def rehash(self):
    """Rehash the hash table when the load factor is too high."""
    old_table = self.table
    self.size = self.size * 2 # Double the size
    self.table = [None] * self.size
    self.count = 0

    # Reinsert all items into the new table
    for item in old_table:
        if item is not None:
            key, value = item
            self.insert(key, value)
```

Sets in Python

"Python `set` is also implemented using a hash table! The elements themselves are used as the keys, and there are no values. This is why checking `if item in my_set:` is $O(1)$."

python

```
# Sets use hash tables internally
my_set = {1, 2, 3, 4, 5}

# Membership check is O(1) on average
if 3 in my_set:
    print("3 is in the set") # Fast!

# Adding and removing are also O(1)
my_set.add(6)
my_set.remove(2)
```

Practical Application: Word Frequency Counter

python

```
def word_frequency(text):
    """Count the frequency of each word in a text."""
    # Clean and split the text
    words = text.lower().split()

    # Create an empty dictionary
    frequency = {}

    # Count each word
    for word in words:
        # Remove punctuation
        word = word.strip(".,!?")

        if word in frequency:
            frequency[word] += 1
        else:
            frequency[word] = 1

    return frequency

# Example usage
text = "The cat sat on the mat. The cat was happy."
```

```
counts = word_frequency(text)

for word, count in counts.items():
    print(f"{word}: {count}")

# Output:
# the: 3
# cat: 2
# sat: 1
# on: 1
# mat: 1
# was: 1
# happy: 1
```

6. Worksheets

Worksheet	Topic
Worksheet 1	ADTs and Python dictionaries
Worksheet 2	Hashing functions and index calculation
Worksheet 3	Collision resolution (linear probing vs. chaining)
Worksheet 4	Load factor, rehashing, and word frequency

7. Summary: Key Takeaways

Concept	Key Point
ADT	High-level description (what, not how)
Hash Table	Key-value storage with $O(1)$ lookups
Hashing Function	Converts key to array index

Concept	Key Point
Collision	Two keys → same index
Linear Probing	Find next available slot
Chaining	Store multiple items in a list at each slot
Load Factor	Items / Slots
Rehashing	Enlarge table when load factor is too high
Python <code>dict</code>	Hash table implementation
Python <code>set</code>	Hash table with keys only

text

KEY REMINDERS

- ✓ Hash tables provide $O(1)$ average-time lookups
- ✓ Keys are converted to indices by a hash function
- ✓ Collisions are unavoidable
- ✓ Linear probing: "try the next slot"
- ✓ Chaining: "store a list at each slot"
- ✓ Load factor determines when to rehash
- ✓ Python `dict` and `set` are hash tables

"Hash tables are one of the most powerful data structures—they provide near-instantaneous lookups by using a hash function to convert keys into array indices."